Digital Systems

Lecture 12

Operating Systems

Mark van der Wilk

Hilary Term 2026

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Problems with Previous Programs

```
static int row = 0;
void advance(void) {  // Called by an interrupt
  row++;
  if (row == 3) row = 0;
  GPIO_OUT = heart[row];
}
```

- Efficient but inflexible: No waiting around, but needed to manually add code to the interrupt (what if another piece of code wanted to use timer?)
- No control structures inside `advace()`. Effectively, we had to implement a program counter ourselves (`row`).

Problems with Previous Programs

Use interrupts to overlap printing with the search, but

- When the serial buffer is full, wastes time waiting in a loop.
- Disables interrupts to protect the buffer from concurrent modification – hard to get right.

Problems with Previous Programs

Use interrupts to overlap printing with the search, but

- When the serial buffer is full, wastes time waiting in a loop.
- Disables interrupts to protect the buffer from concurrent modification – hard to get right.

To illustrate inflexibility: Consider writing a program that simultaneously runs beating `heart` and `primes`.

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

-
-
-
-
-

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
-
-
-
-

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
-
-
-

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
-
-

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
- seeming as each program is running simultaneously,
-

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
- seeming as each program is running simultaneously,
- and doesn't require the programmer to know when to disable interrupts in order to guarantee correct code.

Why concurrency?

- Sharing one machine between several tasks
- Responding to several sources of events
- Genuinely parallel machines (modern multicore)
- Decomposing one task clearly

Operating System

In next few lectures: Example design of an Operating System

- Many different ways of designing an OS, with pros/cons.
- micro:bian's design is based on "message passing".
- Other structures are arguably more efficient.
- You will discuss these structures next year (Concurrency), but you *won't* see low-level implementation of what allows concurrent processes to run.
- We will discuss low-level mechanisms for allowing concurrent processes.

OS Constructs

- Processes: embedded programs are conveniently structured as a set of independent processes.
- Messages: processes can cooperate by exchanging messages in a way that synchronises their behaviour.
- Shared variables are best avoided by using messages instead (error-prone, and can be eliminated… at a cost)

OS Constructs

- Processes: embedded programs are conveniently structured as a set of independent processes.
- Messages: processes can cooperate by exchanging messages in a way that synchronises their behaviour.
- Shared variables are best avoided by using messages instead (error-prone, and can be eliminated… at a cost)

In this lecture, we use these concepts to build a concurrent program.

A Process: Heart

```
static void heart_task(int arg) {
  while (1) {
    show(heart, 70); show(small, 10);
    show(heart, 10); show(small, 10);
  }
}
void show(const unsigned *img, int n){
  while (n-- > 0) {  /* Takes 15msec per iteration */
    for (int p = 0; p < 3; p++) {
      GPIO_OUT = img[p]; timer_delay(5);
    }
} }
```

Another Process: Prime

```
static void prime_task(int arg) {
  int p = 2, n = 0;
  while (1) {
    if (prime(p)) {
      n++;
      // printf will yield by calling serial_putc()
      printf("prime(%d) = %d\n", n, p);
    }
  }
  p++;
}
```

Processes

- Run until no more progress can be made
  (co-operative multitasking vs pre-emptive multitasking)
- When this happens, process *yields* control of CPU, so other process can make progress.
- OS will decide which process to devote time to next.
- After 5ms, “operating system” wakes up process again, and continues.
- Interrupts can still occur, and interrupt a process.

Notice:

- Yield could happen in a subroutine.
- So, each process needs its own subroutine stack.

Startup

```
void init(void) {
  SERIAL = start("Serial", serial_task, 0, STACK);
  TIMER = start("Timer", timer_task, 0, STACK);
  HEART = start("Heart", heart_task, 0, STACK);
  PRIME = start("Prime", prime_task, 0, STACK);
}
```

- a fixed collection of processes created before concurrent execution begins.
- our two processes, plus device drivers for the timer (`timer_delay`) and serial port (`serial_putc`);
- plus an idle task: Only place where CPU gets put to sleep.

Processes

We saw:

- Each process is a main program in its own right
- It can pause itself (or be interrupted) at any point to give others a go.

We will see implementation:

- Processes have interleaved access to CPU
- Each has its own stack

micro:bian supports a fixed set of processes.

? How can processes communicate amongst them?

Let's rewrite the prime example to split calculating, and handing the "UI" (printing to terminal).

Sending Messages

```
void prime_task(int arg) { int n = 2;
  message m;
  while (1) {
    if (prime(n)) {
      m.int1 = n;
      send(USEPRIME, PRIME, &m);
    }
    n++;
  }
}
```

Receiving Messages

```
void summary_task(int arg) {
  int count = 0, limit = arg; message m;
  while (1) {
    receive(PRIME, &m);
    while (m.int1 >= limit) {
      printf("There are %d primes less than %d\n",
             count, limit);
      limit += arg;
    }
    count++;
  }
}
```

Message Communication

```
void init(void) {
  ...
  GENPRIME = start("GenPrime", prime_task, 0, STACK);
  USEPRIME = start("UsePrime", summary_task, 1000, STACK);
}
```

- Messages not buffered.
- If process wants to receive: It waits for the other process to send.
- If process wants to send: It waits for other process to receive.
- If message not ready, process yields, and OS runs another process.
- If message **is** ready, OS transfers when both processes are frozen.
- If two processes send to same receiver, OS determines ordering.

Message Format

Both sender and receiver have a message buffer (16 bytes).

- Message type (2 bytes, `short`)
- Sender (2 bytes, `short`)
- Data (3 `ints`, 12 bytes)

Alternatives to messages

Message passing:

- no “shared variables” between processes.
- all communication by messages

Shared variables with semaphores:

- like the serial output buffer.
- more efficient, but hard to get right.

Other OSs

- Processes with communication
- Drivers for I/O devices
- Dynamically and unloading processes
- Memory management (protection/segmentation, virtual memory)
- File system
- Networking

Summary

- Why we need an OS
- How independent processes operate
- How processes can communicate by messages
- Danger of shared variables, safetey of messages

Example

Heart & Primes example.

Sequence of events, and when CPU sleeps, and wakes.